# Clinic Appointment Booking API

## Full implementation and design rationale

Clinics publish time slots; patients book them. A slot can never be double-booked, even when two requests arrive at the same moment.

| | |
|---|---|
| Stack | Python 3.11+, FastAPI, SQLAlchemy 2.0, SQLite (PostgreSQL supported), pytest |
| Endpoints | 12, across clinics, slots and bookings |
| Tests | 31 passing, including simultaneous-request tests |
| Date | 22 October 2026 |

### Contents


1. Overview
2. Tech stack and why
3. Project structure and why
4. Data model and why
5. How double-booking is prevented, and why this way
6. Full source code
7. API reference and example requests
8. Automated tests and why these tests
9. README: setup, run and test
10. Trade-offs, limitations and next steps

# 1. Overview

This is a FastAPI + SQLAlchemy service where clinics publish time slots and patients book them. The database, not application code, is what guarantees that a slot is never booked twice, even when two requests arrive at the same instant.

Every rule below is enforced in one place and has a test. The "why" for each choice sits in the section that introduces it.

| Rule | How it is enforced | Response when broken |
|---|---|---|
| A slot has at most one active booking, even under simultaneous requests | Atomic conditional `UPDATE` that claims the slot, plus a partial unique index as a second guard | 409 `slot_already_booked` |
| A cancelled slot cannot be booked | The claim only succeeds when the slot status is `open` | 409 `slot_cancelled` |
| A slot that has already started cannot be booked, and a new slot must start in the future | Time check inside the same atomic claim; schema validation on create | 409 `slot_in_past` on booking, 422 on create |
| Cancelling a booking frees the slot for someone else | Booking and slot change in one transaction | 200 |
| Cancelling a slot also cancels its active booking | Slot and booking change in one transaction | 200 |
| A booking or slot cannot be cancelled twice | Conditional `UPDATE` on the current status | 409 `booking_already_cancelled`, 409 `slot_already_cancelled` |
| Input is valid: end after start, 5 minute to 8 hour duration, timezone-aware times, real email address | Pydantic schemas | 422 `validation_error` with a field-by-field list |

## Headline decisions

- **Language and framework:** Python 3.11+ with FastAPI. Request validation and OpenAPI docs come for free, and the code stays short enough to review.
- **Database:** SQLite by default so the reviewer needs no setup; PostgreSQL by changing one environment variable. The concurrency design works identically on both.
- **Structure:** routers (HTTP only), services (business rules), models (tables). Rules can be tested without HTTP, and HTTP can be changed without touching rules.
- **Errors:** one JSON shape for every failure, with a stable machine-readable `code` and a human-readable `message`.
- **Cancel means soft cancel:** rows are kept with a `cancelled` status, so history and audit are preserved.

# 2. Tech stack and why

The stack is chosen to make the booking rules easy to get right and easy for a reviewer to run in under two minutes. Nothing here is exotic on purpose.

| Choice | Why we use it | What we rejected, and why |
|---|---|---|
| Python 3.11+ | Short, readable code, so reviewers spend their time on the rules instead of boilerplate. | None needed; any language could do this. |
| FastAPI | Pydantic validation turns bad input into precise 422 errors with no hand-written checks. It also serves interactive docs at `/docs`, so a reviewer can try every endpoint in a browser. Dependency injection lets tests swap the clock and database. | Django REST Framework: heavier setup for a small API. Express: validation and typing need extra libraries. |
| Synchronous endpoints (`def`, not `async def`) | FastAPI runs them in a thread pool, so requests really do overlap, which is exactly what the concurrency tests need. The guarantee lives in the database, so async adds complexity and no extra correctness. | An async driver: more moving parts for no gain at this scale. |

| Choice | Why we use it | What we rejected, and why |
|---|---|---|
| SQLAlchemy 2.0 | One code base runs on SQLite and PostgreSQL. Its `update()` returns `rowcount`, which is how we detect who won a race. | Raw SQL strings: no portability, easier to get wrong. |
| SQLite by default | Zero setup (one file). It supports partial unique indexes, which the second safety net needs, and serialises writers, which keeps races safe. | Requiring Postgres to run the project: a barrier for reviewers. |
| PostgreSQL (opt-in) | Row-level locks and high write concurrency for production. Switch by setting `DATABASE_URL`; no code changes. | None. This is the production target. |
| Pydantic `EmailStr`, `AwareDatetime` | Real email checking and a hard ban on timezone-less times, which are the classic cause of off-by-hours booking bugs. | Regex email checks and naive datetimes. |
| pytest, `TestClient`, injected clock | Fast tests with no running server. A fake clock tests "slot already started" without sleeping. A real SQLite file per test lets many threads share one database. | In-memory SQLite: each connection gets its own empty database, which breaks multi-threaded tests. |

# 3. Project structure and why

Code is split into three layers so each rule lives in exactly one place and can be tested on its own.

```
clinic-booking/
|-- app/
|   |-- main.py          # app factory: wires database, clock, routers, error handlers
|   |-- config.py        # database URL and business constants
|   |-- database.py      # engine creation, SQLite settings
|   |-- models.py        # tables, constraints, the partial unique index
|   |-- schemas.py       # request/response shapes and field validation
|   |-- errors.py        # domain errors and the one JSON error format
|   |-- deps.py          # per-request database session and clock
|   |-- routers/         # HTTP only: parse request, call a service, return JSON
|   |   |-- clinics.py
|   |   |-- slots.py
|   |   `-- bookings.py
|   `-- services/        # business rules: no HTTP, no FastAPI
|       |-- clinics.py
|       |-- slots.py
|       `-- bookings.py
|-- tests/
|   |-- conftest.py      # fixtures: temp database, fake clock, helpers
|   |-- test_slots.py
|   |-- test_bookings.py
|   `-- test_concurrency.py
|-- requirements.txt
|-- pytest.ini
`-- README.md
```

| Layer | Responsibility | Why it is separate |
|---|---|---|
| Routers | Turn HTTP into function calls and results into JSON. | Changing a URL or status code never risks a business rule. |
| Services | Own every booking rule and every database write. | Rules can be called from tests, a CLI or a queue worker with no web server. The concurrency tests call services directly. |
| Models and schemas | Models describe storage; schemas describe the API. | The API can change shape without a database migration, and the reverse. |
| Errors | Services raise `ConflictError`, `NotFoundError`, `InvalidInputError`. One handler turns them into JSON. | Every error has the same shape, and services stay free of HTTP details. |
| App factory (`create_app`) | Builds a fresh app with its own database and clock. | Each test gets an isolated database. No global state to leak between tests. |

A booking request flows through the layers like this:

1. The router validates the path id and the JSON body (Pydantic).
2. The router calls `bookings.book_slot(db, slot_id, data, now)`.
3. The service runs one atomic claim on the slot (section 5), then inserts the booking.

4. On success the router returns 201 with the booking. On a rule failure the service raises a domain error and the handler returns 404 or 409 with a clear code.

# 4. Data model and why

Three tables hold everything. The schema rejects invalid states by itself, so a future bug in application code cannot quietly corrupt bookings.

| Table | Columns | Database-level rules | Why |
|---|---|---|---|
| `clinics` | `id`, `name`, `created_at` | Primary key | Gives slots a real owner, so an unknown clinic returns 404 instead of creating orphan data. |
| `slots` | `id`, `clinic_id`, `start_time`, `end_time`, `status` (open, booked, cancelled), `created_at` | Foreign key to clinics; `end_time > start_time`; status must be one of three values; index on (`clinic_id`, `start_time`) | `status` on the slot is what the atomic claim flips. The index makes "list a clinic's slots by time" fast. |
| `bookings` | `id`, `slot_id`, `patient_name`, `patient_email`, `status` (confirmed, cancelled), `created_at`, `cancelled_at` | Foreign key to slots; status check; **unique index on `slot_id` where `status = 'confirmed'`** | Two confirmed bookings for one slot cannot exist, whatever the code does. Cancelled rows are exempt, so a slot can be rebooked after a cancellation. |

## Decisions behind the schema

- **Status lives on both slot and booking.** The slot status is the single value that requests compete for, which is what makes the atomic claim possible. The booking status keeps history. The invariant is: a slot is `booked` if and only if it has exactly one `confirmed` booking. Every code path changes both in one transaction, and a concurrency test checks the invariant.
- **Soft cancel, never delete.** A cancelled slot or booking keeps its row with `cancelled` status, preserving an audit trail.
- **Times are stored in UTC and returned with a `Z` suffix.** The custom `UTCDateTime` type refuses naive datetimes and converts everything on the way in and out, identically on SQLite and PostgreSQL.
- **No patient table.** The brief has no accounts or login, so a patient is a name and an email stored on the booking. Emails are lowercased so lookups are case-insensitive.
- **Foreign keys are switched on explicitly for SQLite**, which ignores them by default.

## Allowed state changes

| Object | From | Event | To | Guard (checked atomically) |
|---|---|---|---|---|
| Slot | `open` | Patient books | `booked` | Slot is `open` and has not started |
| Slot | `booked` | Patient cancels booking | `open` | The booking was still `confirmed`; slot has not started |
| Slot | `open` or `booked` | Clinic cancels slot | `cancelled` | Slot has not started; its confirmed booking is cancelled in the same transaction |
| Slot | `cancelled` | Anything | Not allowed | Final state |
| Booking | `confirmed` | Patient cancels, or clinic cancels the slot | `cancelled` | Still `confirmed`; slot has not started |
| Booking | `cancelled` | Anything | Not allowed | Final state |

# 5. How double-booking is prevented, and why this way

A slot is claimed with one conditional `UPDATE`, which the database executes atomically, so out of any number of simultaneous requests exactly one succeeds. This is the most important design decision in the project.

## Why the obvious approach fails

The natural code is "read the slot, check it is open, then mark it booked". Between the read and the write there is a gap, and two requests can both fall into it:

| Step | Request A | Request B |
|---|---|---|
| 1 | Reads slot: `open` | |
| 2 | | Reads slot: `open` |
| 3 | Writes `booked`, inserts booking | |
| 4 | | Writes `booked`, inserts booking |
| Result | Two bookings for one slot | |

This bug is invisible in manual testing because it needs two requests within milliseconds. We proved it is real: when we replaced the claim with this naive version, 15 of 20 simultaneous requests all "won" the same slot.

## The fix: check and write in one statement

```
UPDATE slots
   SET status = 'booked'
 WHERE id = :slot_id
   AND status = 'open'
   AND start_time > :now;
```

The check (`status = 'open'`, not started) and the write happen as one indivisible step. The service then looks at `rowcount`:

- **1 row changed:** this request owns the slot. It inserts the booking and commits.
- **0 rows changed:** someone else got there first, or the slot is cancelled, or it has started. The service rolls back, looks at the slot to choose the right error, and returns 404 or 409.

On SQLite, writers take turns because the database allows one writer at a time. On PostgreSQL, the second `UPDATE` waits for the first transaction's row lock, then re-checks the `WHERE` clause against the committed row, sees `booked`, and matches zero rows. Both give the same outcome.

## Four layers of protection

| Layer | What it does | Why it is there |
|---|---|---|
| 1. Atomic claim (`UPDATE ... WHERE status = 'open'`) | Only one request can flip the slot. | The primary guarantee. It also covers cancelled and past slots in the same statement. |
| 2. Partial unique index on confirmed bookings | The database refuses a second confirmed booking for a slot. | Safety net: if someone later adds a code path that skips the claim, the data still cannot be corrupted. A test inserts two rows directly to prove it. |
| 3. One transaction per operation | Slot status and booking row change together or not at all. | A crash between the two steps can never leave a slot marked booked with no booking. |
| 4. Fixed lock order: slot first, then booking | Booking, cancel-booking and cancel-slot all lock the slot row before the booking row. | Prevents deadlocks on PostgreSQL when a patient cancels while the clinic cancels the slot. |

## Alternatives we rejected

- **A lock in application memory** (a Python `threading.Lock`): works for one process only. Production runs several worker processes, and the lock would protect none of them from each other.
- **A distributed lock (Redis):** adds a new service and new failure modes (expired locks, outages) to solve a problem the database already solves.
- `SELECT ... FOR UPDATE` **then update:** correct on PostgreSQL, but SQLite ignores it, so the same code would behave differently on the two databases.
- **SERIALIZABLE isolation with retries:** correct, but every endpoint would need retry logic for serialization failures. The single-statement claim needs none.

- **Relying only on the unique index and catching the error:** it cannot tell the caller why a booking failed (cancelled, past, already booked) and cannot protect a slot from being booked just after a clinic cancels it.

### Evidence

- A test releases 20 threads at the same instant with a barrier. Exactly 1 booking succeeds and 19 get `slot_already_booked`.
- The same race through HTTP returns one 201 and nineteen 409s.
- A test races "book" against "cancel slot" across 15 slots and checks the invariant afterwards: a cancelled slot never keeps a confirmed booking.
- The concurrency tests ran 25 times in a row on SQLite with no failures. PostgreSQL was not available where this was built and tested, so its behaviour rests on documented READ COMMITTED row-lock semantics rather than a test run; section 10 lists running the suite against Postgres as a next step.

# 6. Full source code

This is the complete application, exactly as it was run and tested (31 tests passing). Each file starts with the reason it exists. Empty `__init__.py` files are needed in `app/`, `app/routers/`, `app/services/` and `tests/`.

## Part A: configuration, storage and validation

### `requirements.txt`

Minimum versions only, so it installs cleanly on any current Python 3.11+.

```
fastapi>=0.110
uvicorn[standard]>=0.29
sqlalchemy>=2.0
pydantic[email]>=2.6
httpx>=0.27
pytest>=8.0
# Only needed if you run against PostgreSQL:
# psycopg[binary]>=3.1
```

### `app/config.py`

Business limits and the database URL live in one place instead of being scattered as magic numbers.

```
"""Settings and business constants in one place."""
import os

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./clinic.db")

MIN_SLOT_MINUTES = 5
MAX_SLOT_MINUTES = 8 * 60
MAX_ID = 2**31 - 1  # fits a PostgreSQL integer; stops overflow errors on huge ids
```

### `app/database.py`

One place that knows how to build an engine. The SQLite branch switches on foreign keys, enables WAL so readers do not block the writer, and sets a 30 second wait so competing writers queue instead of failing with "database is locked".

```
"""Engine creation. SQLite is the default; any SQLAlchemy URL (e.g. PostgreSQL) also works."""
from sqlalchemy import create_engine, event
from sqlalchemy.engine import Engine
from sqlalchemy.orm import DeclarativeBase


class Base(DeclarativeBase):
    pass


def make_engine(url: str) -> Engine:
    if not url.startswith("sqlite"):
        return create_engine(url, pool_pre_ping=True)

    # timeout: wait for a competing writer instead of failing with "database is locked".
    # check_same_thread: FastAPI runs sync endpoints in a thread pool.
    engine = create_engine(url, connect_args={"check_same_thread": False, "timeout": 30})

    @event.listens_for(engine, "connect")
    def _configure(dbapi_connection, _record):
        cursor = dbapi_connection.cursor()
        cursor.execute("PRAGMA foreign_keys=ON")   # SQLite ignores foreign keys by default
        cursor.execute("PRAGMA journal_mode=WAL")  # readers do not block the writer
        cursor.close()

    return engine
```

### `app/models.py`

The tables carry the rules that must never be broken (valid statuses, end after start, one confirmed booking per slot). `UTCDateTime` guarantees every stored time is UTC and every returned time is timezone-aware.

```
"""Tables. The schema itself refuses invalid states, so a bug in code cannot corrupt data."""
from datetime import datetime, timezone

from sqlalchemy import CheckConstraint, DateTime, ForeignKey, Index, String, text
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy.types import TypeDecorator

from app.database import Base


class UTCDateTime(TypeDecorator):
    """Store UTC, return timezone-aware UTC. Works the same on SQLite and PostgreSQL."""
    impl = DateTime
    cache_ok = True

    def process_bind_param(self, value, dialect):
        if value is None:
            return None
        if value.tzinfo is None:
            raise ValueError("naive datetime is not allowed; supply a timezone")
        return value.astimezone(timezone.utc).replace(tzinfo=None)

    def process_result_value(self, value, dialect):
        return None if value is None else value.replace(tzinfo=timezone.utc)


class SlotStatus:
    OPEN = "open"
    BOOKED = "booked"
    CANCELLED = "cancelled"


class BookingStatus:
    CONFIRMED = "confirmed"
    CANCELLED = "cancelled"


class Clinic(Base):
    __tablename__ = "clinics"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(200))
    created_at: Mapped[datetime] = mapped_column(UTCDateTime)


class Slot(Base):
    __tablename__ = "slots"
    __table_args__ = (
        CheckConstraint("end_time > start_time", name="ck_slot_end_after_start"),
        CheckConstraint("status IN ('open', 'booked', 'cancelled')", name="ck_slot_status"),
        Index("ix_slots_clinic_start", "clinic_id", "start_time"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    clinic_id: Mapped[int] = mapped_column(ForeignKey("clinics.id"))
    start_time: Mapped[datetime] = mapped_column(UTCDateTime)
    end_time: Mapped[datetime] = mapped_column(UTCDateTime)
    status: Mapped[str] = mapped_column(String(16), default=SlotStatus.OPEN)
    created_at: Mapped[datetime] = mapped_column(UTCDateTime)


class Booking(Base):
    __tablename__ = "bookings"
    __table_args__ = (
        CheckConstraint("status IN ('confirmed', 'cancelled')", name="ck_booking_status"),
        # Second line of defence: at most ONE confirmed booking per slot, enforced by the database.
        Index(
            "uq_one_confirmed_booking_per_slot", "slot_id", unique=True,
            sqlite_where=text("status = 'confirmed'"),
            postgresql_where=text("status = 'confirmed'"),
        ),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    slot_id: Mapped[int] = mapped_column(ForeignKey("slots.id"), index=True)
    patient_name: Mapped[str] = mapped_column(String(200))
    patient_email: Mapped[str] = mapped_column(String(320), index=True)
    status: Mapped[str] = mapped_column(String(16), default=BookingStatus.CONFIRMED)
    created_at: Mapped[datetime] = mapped_column(UTCDateTime)
    cancelled_at: Mapped[datetime | None] = mapped_column(UTCDateTime, nullable=True)
```

### `app/errors.py`

Services raise meaningful errors; one handler turns them all into the same JSON shape. Clients branch on the stable `code` and show the human `message`.

```
"""Domain errors and the single JSON error format the API returns.

Services raise these; they know nothing about HTTP beyond a status code.
Every failure is rendered as:
    {"error": {"code": "...", "message": "...", "details": [...]}}
"""
from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException


class DomainError(Exception):
    status_code = 400

    def __init__(self, code: str, message: str, details: list[dict] | None = None):
        super().__init__(message)
        self.code = code
        self.message = message
        self.details = details


class NotFoundError(DomainError):
    status_code = 404


class ConflictError(DomainError):
    """The request is valid but clashes with the current state (e.g. slot already booked)."""
    status_code = 409


class InvalidInputError(DomainError):
    """Input that passes schema validation but breaks a rule that needs the clock or the database."""
    status_code = 422

    def __init__(self, field: str, message: str):
        super().__init__("validation_error", "Request validation failed.",
                         [{"field": field, "message": message}])


def _body(code: str, message: str, details: list[dict] | None = None) -> dict:
    error = {"code": code, "message": message}
    if details:
        error["details"] = details
    return {"error": error}


def register_error_handlers(app: FastAPI) -> None:
    @app.exception_handler(DomainError)
    async def domain_error(_: Request, exc: DomainError):
        return JSONResponse(_body(exc.code, exc.message, exc.details), status_code=exc.status_code)

    @app.exception_handler(RequestValidationError)
    async def validation_error(_: Request, exc: RequestValidationError):
        details = [
            {
                "field": ".".join(str(part) for part in err["loc"][1:]) or str(err["loc"][0]),
                "message": err["msg"].removeprefix("Value error, "),
            }
            for err in exc.errors()
        ]
        return JSONResponse(_body("validation_error", "Request validation failed.", details),
                            status_code=422)

    @app.exception_handler(StarletteHTTPException)
    async def http_error(_: Request, exc: StarletteHTTPException):
        return JSONResponse(_body("http_error", str(exc.detail)), status_code=exc.status_code)
```

### `app/schemas.py`

All field-level validation in one file. `extra="forbid"` makes typos fail loudly, `AwareDatetime` bans ambiguous times, and the slot validator enforces end-after-start and duration limits.

```
"""Request and response shapes. Field-level validation lives here."""
from datetime import datetime
from typing import Literal

from pydantic import (AwareDatetime, BaseModel, ConfigDict, EmailStr, Field,
                      model_validator)

from app.config import MAX_SLOT_MINUTES, MIN_SLOT_MINUTES

SlotStatusLiteral = Literal["open", "booked", "cancelled"]
BookingStatusLiteral = Literal["confirmed", "cancelled"]


class _In(BaseModel):
    # Reject unknown fields so typos like "start_tme" fail loudly instead of being ignored.
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)


class _Out(BaseModel):
    model_config = ConfigDict(from_attributes=True)


class ClinicCreate(_In):
    name: str = Field(min_length=1, max_length=200)


class ClinicOut(_Out):
    id: int
    name: str
    created_at: datetime


class SlotCreate(_In):
    # AwareDatetime rejects "2030-01-01T10:00:00" (no timezone), which would be ambiguous.
    start_time: AwareDatetime
    end_time: AwareDatetime

    @model_validator(mode="after")
    def check_range(self):
        if self.end_time <= self.start_time:
            raise ValueError("end_time must be after start_time")
        minutes = (self.end_time - self.start_time).total_seconds() / 60
        if not MIN_SLOT_MINUTES <= minutes <= MAX_SLOT_MINUTES:
            raise ValueError(
                f"slot duration must be between {MIN_SLOT_MINUTES} and {MAX_SLOT_MINUTES} minutes")
        return self
class SlotOut(_Out):
    id: int
    clinic_id: int
    start_time: datetime
    end_time: datetime
    status: SlotStatusLiteral
    created_at: datetime


class BookingCreate(_In):
    patient_name: str = Field(min_length=1, max_length=200)
    patient_email: EmailStr


class BookingOut(_Out):
    id: int
    slot_id: int
    patient_name: str
    patient_email: str
    status: BookingStatusLiteral
    created_at: datetime
    cancelled_at: datetime | None
```

## Part B: business rules and the HTTP layer

### `app/services/bookings.py`

The heart of the project. `book_slot` claims the slot in a single atomic statement, so there is no gap for a second request to slip into. `cancel_booking` re-opens the slot only if its own cancellation succeeded, so a replayed cancel can never reopen a slot that someone else has since booked.

```
"""Booking rules, including the part that makes double-booking impossible.

The key idea: never "check, then write". Claiming the slot is ONE conditional UPDATE:

    UPDATE slots SET status = 'booked'
     WHERE id = :id AND status = 'open' AND start_time > :now

The database applies it atomically, so of any number of simultaneous requests exactly one sees
rowcount == 1. Everyone else sees 0 and is told why.
"""
from datetime import datetime

from sqlalchemy import select, update
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.errors import ConflictError, NotFoundError
from app.models import Booking, BookingStatus, Slot, SlotStatus
from app.schemas import BookingCreate


def book_slot(db: Session, slot_id: int, data: BookingCreate, now: datetime) -> Booking:
    claimed = db.execute(
        update(Slot)
        .where(Slot.id == slot_id, Slot.status == SlotStatus.OPEN, Slot.start_time > now)
        .values(status=SlotStatus.BOOKED)
        .execution_options(synchronize_session=False)
    )
    if claimed.rowcount != 1:
        db.rollback()  # release the write lock before diagnosing
        raise _why_not_bookable(db, slot_id)

    booking = Booking(slot_id=slot_id, patient_name=data.patient_name,
                      patient_email=data.patient_email.lower(), created_at=now)
    db.add(booking)
    try:
        db.commit()
    except IntegrityError:
        # Unreachable while the claim above is correct. The partial unique index is the safety
        # net: if a future code path ever bypasses the claim, the database still refuses.
        db.rollback()
        raise ConflictError("slot_already_booked", "This slot has just been booked by someone else.")
    return booking


def _why_not_bookable(db: Session, slot_id: int) -> Exception:
    slot = db.get(Slot, slot_id)
    if slot is None:
```

```
        return NotFoundError("slot_not_found", f"Slot {slot_id} does not exist.")
    if slot.status == SlotStatus.BOOKED:
        return ConflictError("slot_already_booked", "This slot has already been booked.")
    if slot.status == SlotStatus.CANCELLED:
        return ConflictError("slot_cancelled", "This slot was cancelled by the clinic.")
    return ConflictError("slot_in_past", "This slot has already started and can no longer be booked.")


def get_booking(db: Session, booking_id: int) -> Booking:
    booking = db.get(Booking, booking_id)
    if booking is None:
        raise NotFoundError("booking_not_found", f"Booking {booking_id} does not exist.")
    return booking


def cancel_booking(db: Session, booking_id: int, now: datetime) -> Booking:
    """Cancel a booking and re-open its slot, atomically."""
    booking = get_booking(db, booking_id)

    # Lock the slot first (same order as book_slot and cancel_slot) to avoid deadlocks.
    # SQLite ignores FOR UPDATE; it serialises writers on its own.
    slot = db.scalars(
        select(Slot).where(Slot.id == booking.slot_id).with_for_update()
        .execution_options(populate_existing=True)
    ).one()
    if slot.start_time <= now:
        db.rollback()
        raise ConflictError("slot_in_past", "A booking cannot be cancelled after its slot has started.")

    cancelled = db.execute(
        update(Booking)
        .where(Booking.id == booking_id, Booking.status == BookingStatus.CONFIRMED)
        .values(status=BookingStatus.CANCELLED, cancelled_at=now)
        .execution_options(synchronize_session=False)
    )
    if cancelled.rowcount != 1:
        db.rollback()
        raise ConflictError("booking_already_cancelled", "This booking is already cancelled.")

    db.execute(
        update(Slot)
        .where(Slot.id == slot.id, Slot.status == SlotStatus.BOOKED)
        .values(status=SlotStatus.OPEN)
        .execution_options(synchronize_session=False)
    )
    db.commit()
    return get_booking(db, booking_id)
def list_bookings(db: Session, *, patient_email: str | None = None, clinic_id: int | None = None,
                  status: str | None = None, limit: int, offset: int) -> list[Booking]:
    query = select(Booking)
    if clinic_id is not None:
        query = query.join(Slot, Slot.id == Booking.slot_id).where(Slot.clinic_id == clinic_id)
    if patient_email is not None:
        query = query.where(Booking.patient_email == patient_email.lower())
    if status:
        query = query.where(Booking.status == status)
    return list(db.scalars(query.order_by(Booking.id).limit(limit).offset(offset)))
```

### `app/services/slots.py`

Slot rules in one place. Cancelling uses the same claim pattern as booking, and cancels the slot's booking in the same transaction so the two can never disagree.

```
"""Slot rules. Cancelling a slot is a state change, never a delete."""
from datetime import datetime

from sqlalchemy import select, update
from sqlalchemy.orm import Session

from app.errors import ConflictError, InvalidInputError, NotFoundError
from app.models import Booking, BookingStatus, Slot, SlotStatus
from app.schemas import SlotCreate
from app.services.clinics import get_clinic


def create_slot(db: Session, clinic_id: int, data: SlotCreate, now: datetime) -> Slot:
    get_clinic(db, clinic_id)  # 404 if the clinic does not exist
    if data.start_time <= now:
        raise InvalidInputError("start_time", "start_time must be in the future")
    slot = Slot(clinic_id=clinic_id, start_time=data.start_time,
                end_time=data.end_time, created_at=now)
    db.add(slot)
    db.commit()
    return slot


def get_slot(db: Session, slot_id: int) -> Slot:
    slot = db.get(Slot, slot_id)
    if slot is None:
        raise NotFoundError("slot_not_found", f"Slot {slot_id} does not exist.")
    return slot


def list_slots(db: Session, clinic_id: int, status: str | None,
               starts_after: datetime | None, starts_before: datetime | None,
               limit: int, offset: int) -> list[Slot]:
    get_clinic(db, clinic_id)
    query = select(Slot).where(Slot.clinic_id == clinic_id)
    if status:
        query = query.where(Slot.status == status)
    if starts_after:
        query = query.where(Slot.start_time >= starts_after)
    if starts_before:
        query = query.where(Slot.start_time < starts_before)
    return list(db.scalars(query.order_by(Slot.start_time, Slot.id).limit(limit).offset(offset)))


def cancel_slot(db: Session, slot_id: int, now: datetime) -> Slot:
    """Cancel a slot and, in the same transaction, any confirmed booking on it.

    The slot is claimed first (slot lock, then booking lock). Every flow that touches both rows
    uses this order, which is what prevents deadlocks on PostgreSQL.
    """
    claimed = db.execute(
        update(Slot)
        .where(Slot.id == slot_id, Slot.status.in_([SlotStatus.OPEN, SlotStatus.BOOKED]),
               Slot.start_time > now)
        .values(status=SlotStatus.CANCELLED)
        .execution_options(synchronize_session=False)
    )
    if claimed.rowcount != 1:
        db.rollback()  # release the write lock before looking at why it failed
        slot = get_slot(db, slot_id)
        if slot.status == SlotStatus.CANCELLED:
            raise ConflictError("slot_already_cancelled", "This slot is already cancelled.")
        raise ConflictError("slot_in_past", "A slot that has already started cannot be cancelled.")

    db.execute(
        update(Booking)
        .where(Booking.slot_id == slot_id, Booking.status == BookingStatus.CONFIRMED)
        .values(status=BookingStatus.CANCELLED, cancelled_at=now)
        .execution_options(synchronize_session=False)
    )
    db.commit()
    return get_slot(db, slot_id)
```

### app/services/clinics.py

Clinics own slots; a tiny service keeps the "does this clinic exist" check reusable.

```
from datetime import datetime

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.errors import NotFoundError
from app.models import Clinic
from app.schemas import ClinicCreate


def create_clinic(db: Session, data: ClinicCreate, now: datetime) -> Clinic:
    clinic = Clinic(name=data.name, created_at=now)
    db.add(clinic)
    db.commit()
    return clinic


def get_clinic(db: Session, clinic_id: int) -> Clinic:
    clinic = db.get(Clinic, clinic_id)
    if clinic is None:
        raise NotFoundError("clinic_not_found", f"Clinic {clinic_id} does not exist.")
    return clinic


def list_clinics(db: Session, limit: int, offset: int) -> list[Clinic]:
    return list(db.scalars(select(Clinic).order_by(Clinic.id).limit(limit).offset(offset)))
```

### app/deps.py

One session per request, and the clock injected rather than read directly, so tests can check "slot already started" without sleeping.

```
"""FastAPI dependencies: a database session and the current time, both per request."""
from datetime import datetime
from typing import Annotated, Iterator

from fastapi import Depends, Path, Query, Request
from sqlalchemy.orm import Session

from app.config import MAX_ID

Id = Annotated[int, Path(ge=1, le=MAX_ID)]
Limit = Annotated[int, Query(ge=1, le=100)]
Offset = Annotated[int, Query(ge=0)]


def get_db(request: Request) -> Iterator[Session]:
    with request.app.state.session_factory() as session:
        yield session


def get_now(request: Request) -> datetime:
    # The clock is injected so tests can control time instead of sleeping or patching.
    return request.app.state.clock()


Db = Annotated[Session, Depends(get_db)]
Now = Annotated[datetime, Depends(get_now)]
```

### app/routers/clinics.py

HTTP only. Each function parses the request, calls one service function and returns the result. No business rule lives here.

```
from typing import Annotated

from fastapi import APIRouter, Query
from pydantic import AwareDatetime

from app.deps import Db, Id, Limit, Now, Offset
from app.schemas import (BookingOut, BookingStatusLiteral, ClinicCreate, ClinicOut, SlotCreate,
                         SlotOut, SlotStatusLiteral)
from app.services import bookings, clinics, slots

router = APIRouter(prefix="/clinics", tags=["clinics"])


@router.post("", response_model=ClinicOut, status_code=201)
def create_clinic(body: ClinicCreate, db: Db, now: Now):
    return clinics.create_clinic(db, body, now)


@router.get("", response_model=list[ClinicOut])
def list_clinics(db: Db, limit: Limit = 50, offset: Offset = 0):
    return clinics.list_clinics(db, limit, offset)


@router.get("/{clinic_id}", response_model=ClinicOut)
def get_clinic(clinic_id: Id, db: Db):
    return clinics.get_clinic(db, clinic_id)


@router.post("/{clinic_id}/slots", response_model=SlotOut, status_code=201)
def create_slot(clinic_id: Id, body: SlotCreate, db: Db, now: Now):
    return slots.create_slot(db, clinic_id, body, now)


@router.get("/{clinic_id}/slots", response_model=list[SlotOut])
def list_slots(clinic_id: Id, db: Db,
               status: Annotated[SlotStatusLiteral | None, Query()] = None,
               starts_after: Annotated[AwareDatetime | None, Query()] = None,
               starts_before: Annotated[AwareDatetime | None, Query()] = None,
               limit: Limit = 50, offset: Offset = 0):
    return slots.list_slots(db, clinic_id, status, starts_after, starts_before, limit, offset)


@router.get("/{clinic_id}/bookings", response_model=list[BookingOut])
def list_clinic_bookings(clinic_id: Id, db: Db,
                         status: Annotated[BookingStatusLiteral | None, Query()] = None,
                         limit: Limit = 50, offset: Offset = 0):
    clinics.get_clinic(db, clinic_id)
    return bookings.list_bookings(db, clinic_id=clinic_id, status=status, limit=limit, offset=offset)
```

### app/routers/slots.py

```
from fastapi import APIRouter

from app.deps import Db, Id, Now
from app.schemas import BookingCreate, BookingOut, SlotOut
from app.services import bookings, slots

router = APIRouter(prefix="/slots", tags=["slots"])


@router.get("/{slot_id}", response_model=SlotOut)
def get_slot(slot_id: Id, db: Db):
    return slots.get_slot(db, slot_id)


@router.post("/{slot_id}/cancel", response_model=SlotOut)
def cancel_slot(slot_id: Id, db: Db, now: Now):
    return slots.cancel_slot(db, slot_id, now)


@router.post("/{slot_id}/bookings", response_model=BookingOut, status_code=201)
def book_slot(slot_id: Id, body: BookingCreate, db: Db, now: Now):
    return bookings.book_slot(db, slot_id, body, now)
```

### app/routers/bookings.py

```
from typing import Annotated

from fastapi import APIRouter, Query
from pydantic import EmailStr

from app.deps import Db, Id, Limit, Now, Offset
from app.schemas import BookingOut, BookingStatusLiteral
from app.services import bookings

router = APIRouter(prefix="/bookings", tags=["bookings"])


@router.get("", response_model=list[BookingOut])
def list_my_bookings(db: Db, patient_email: Annotated[EmailStr, Query()],
                     status: Annotated[BookingStatusLiteral | None, Query()] = None,
                     limit: Limit = 50, offset: Offset = 0):
    return bookings.list_bookings(db, patient_email=patient_email, status=status,
                                  limit=limit, offset=offset)


@router.get("/{booking_id}", response_model=BookingOut)
def get_booking(booking_id: Id, db: Db):
    return bookings.get_booking(db, booking_id)


@router.post("/{booking_id}/cancel", response_model=BookingOut)
def cancel_booking(booking_id: Id, db: Db, now: Now):
    return bookings.cancel_booking(db, booking_id, now)
```

### app/main.py

An application factory, so every test builds a fresh app with its own database and a fake clock. Run it with `--factory`.

```
"""Application factory. Run with:  uvicorn app.main:create_app --factory"""
from datetime import datetime, timezone
from typing import Callable

from fastapi import FastAPI
from sqlalchemy.orm import sessionmaker

from app import config, models  # noqa: F401  (importing models registers the tables)
from app.database import Base, make_engine
from app.errors import register_error_handlers
from app.routers import bookings, clinics, slots


def utcnow() -> datetime:
    return datetime.now(timezone.utc)


def create_app(database_url: str | None = None,
               clock: Callable[[], datetime] = utcnow) -> FastAPI:
    engine = make_engine(database_url or config.DATABASE_URL)
    Base.metadata.create_all(engine)  # use Alembic migrations in production

    app = FastAPI(title="Clinic Booking API", version="1.0.0")
    app.state.engine = engine
    app.state.session_factory = sessionmaker(engine)
    app.state.clock = clock

    register_error_handlers(app)
    for module in (clinics, slots, bookings):
        app.include_router(module.router)
    return app
```

# 7. API reference and example requests

The API has 12 endpoints around three resources. Interactive docs for all of them are served at `/docs` when the app is running.

| Method | Path | Purpose | Success |
|---|---|---|---|
| POST | `/clinics` | Create a clinic | 201 |
| GET | `/clinics` | List clinics (`limit`, `offset`) | 200 |
| GET | `/clinics/{clinic_id}` | Get one clinic | 200 |
| POST | `/clinics/{clinic_id}/slots` | Publish a slot | 201 |
| GET | `/clinics/{clinic_id}/slots` | List slots; filters `status`, `starts_after`, `starts_before`, plus `limit`, `offset` | 200 |
| GET | `/clinics/{clinic_id}/bookings` | List a clinic's bookings; filter `status` | 200 |
| GET | `/slots/{slot_id}` | Get one slot | 200 |
| POST | `/slots/{slot_id}/cancel` | Cancel a slot (also cancels its booking) | 200 |
| POST | `/slots/{slot_id}/bookings` | Book a slot | 201 |
| GET | `/bookings?patient_email=...` | List a patient's bookings; filter `status` | 200 |
| GET | `/bookings/{booking_id}` | Get one booking | 200 |
| POST | `/bookings/{booking_id}/cancel` | Cancel a booking (re-opens the slot) | 200 |

## Why the resources look like this

- **Nested creation.** A slot is created under its clinic and a booking under its slot. The URL says who owns the new thing, and an unknown parent gives a natural 404.
- **Cancel is `POST .../cancel`, not `DELETE`.** We keep the row and change its status, so `DELETE` would promise something untrue. The response returns the updated object.
- **Booking a slot is a `POST` that creates a booking (201).** The slot's status change is a side effect, not something the client sets directly.
- **Lists are paginated** (`limit` 1 to 100, default 50) and slots are ordered by start time, so a response can never be unbounded.
- **Listing a patient's bookings requires `patient_email`.** There is no login in this brief, so the API never offers a "list everyone's bookings" endpoint.

## Status codes

| Code | When | Example error.code |
|---|---|---|
| 201 | A slot, booking or clinic was created | none |
| 200 | Read succeeded, or a cancel succeeded | none |
| 404 | The clinic, slot or booking does not exist | `clinic_not_found`, `slot_not_found`, `booking_not_found` |
| 409 | The request is valid but conflicts with current state | `slot_already_booked`, `slot_cancelled`, `slot_in_past`, `slot_already_cancelled`, `booking_already_cancelled` |
| 422 | The input itself is invalid | `validation_error` |

409 versus 422 is deliberate. 422 means "fix your request"; 409 means "your request was fine, but the world has changed", which is the right signal for a patient who lost a race and should pick another slot.

## Error format

Every error has the same shape. `code` is stable for programs; `message` is for people; `details` lists each invalid field.

```
{
  "error": {
    "code": "validation_error",
    "message": "Request validation failed.",
    "details": [
      {"field": "patient_name", "message": "String should have at least 1 character"},
      {"field": "patient_email", "message": "value is not a valid email address: An email address must have an @-sign."}
    ]
  }
}
```

### Example session

These responses were captured from the running server; `...` marks fields shortened here. Times are ISO 8601 and must include a timezone (`Z` for UTC). In query strings, write `+05:30` as `%2B05:30`, because a bare `+` is read as a space.

```
# 1. Create a clinic
curl -X POST http://127.0.0.1:8000/clinics \
  -H 'Content-Type: application/json' -d '{"name":"Green Valley Clinic"}'
# 201 {"id":1,"name":"Green Valley Clinic","created_at":"2026-10-22T16:34:43.413658Z"}

# 2. Publish a slot
curl -X POST http://127.0.0.1:8000/clinics/1/slots \
  -H 'Content-Type: application/json' \
  -d '{"start_time":"2030-03-01T09:00:00Z","end_time":"2030-03-01T09:30:00Z"}'
# 201 {"id":1,"clinic_id":1,"start_time":"2030-03-01T09:00:00Z",
#      "end_time":"2030-03-01T09:30:00Z","status":"open","created_at":"..."}

# 3. List open slots
curl 'http://127.0.0.1:8000/clinics/1/slots?status=open'
# 200 [ {...the slot above...} ]

# 4. Book it
curl -X POST http://127.0.0.1:8000/slots/1/bookings \
  -H 'Content-Type: application/json' \
  -d '{"patient_name":"Asha Rao","patient_email":"asha@example.com"}'
# 201 {"id":1,"slot_id":1,"patient_name":"Asha Rao","patient_email":"asha@example.com",
#      "status":"confirmed","created_at":"...","cancelled_at":null}

# 5. Someone else tries the same slot
curl -X POST http://127.0.0.1:8000/slots/1/bookings \
  -H 'Content-Type: application/json' \
  -d '{"patient_name":"Ravi Kumar","patient_email":"ravi@example.com"}'
# 409 {"error":{"code":"slot_already_booked","message":"This slot has already been booked."}}

# 6. Invalid input
curl -X POST http://127.0.0.1:8000/slots/1/bookings \
  -H 'Content-Type: application/json' -d '{"patient_name":"","patient_email":"nope"}'
# 422 (the error body shown above)

# 7. Cancel the booking (the slot becomes open again)
curl -X POST http://127.0.0.1:8000/bookings/1/cancel
# 200 {"id":1,...,"status":"cancelled","cancelled_at":"2026-10-22T16:34:43.489839Z"}

# 8. Cancel the slot
curl -X POST http://127.0.0.1:8000/slots/1/cancel
# 200 {"id":1,...,"status":"cancelled",...}

# 9. A patient's bookings
curl 'http://127.0.0.1:8000/bookings?patient_email=asha@example.com'
# 200 [ {... "status":"cancelled" ...} ]
```

# 8. Automated tests and why these tests

There are 31 tests, and every important booking rule has at least one. Tests assert behaviour a client can see (status code, error code, resulting state), not internal details, so they survive refactoring.

| Rule | Test | Why it matters |
|---|---|---|
| One winner among 20 simultaneous bookings | `test_many_simultaneous_bookings_of_one_slot_produce_exactly_one_winner` | The core requirement. A barrier releases all threads together so requests truly collide. |
| Same race through HTTP | `test_simultaneous_http_requests_return_one_201_and_the_rest_409` | Proves the guarantee holds end to end, including the thread pool and error mapping. |
| Book versus cancel-slot race | `test_booking_racing_against_slot_cancellation_never_leaves_inconsistent_state` | Checks the invariant: a cancelled slot never keeps a confirmed booking. |
| Cancel versus rebook race | `test_simultaneous_cancel_and_rebook_keep_slot_and_booking_in_step` | Covers the freed-slot path, where state is most likely to drift. |
| Database refuses a second confirmed booking | `test_database_itself_refuses_a_second_confirmed_booking` | Proves the safety net works even if the service layer is bypassed. |
| Cancelled rows do not block rebooking | `test_database_allows_a_new_booking_once_the_old_one_is_cancelled` | Guards against making the index too strict. |
| Booked, cancelled and started slots are refused | `test_a_booked_slot_...`, `test_a_cancelled_slot_...`, `test_a_slot_that_has_started_...` | Each gives a different, specific 409 code. |
| Cancel frees the slot; cancel twice is refused | `test_cancelling_a_booking_frees_...`, `test_a_booking_cannot_be_cancelled_twice` | Lifecycle correctness. |
| A stale cancel cannot reopen a rebooked slot | `test_a_stale_cancel_does_not_reopen_...` | A subtle bug naive code has: replaying an old cancel would free someone else's booking. |
| Slot cancel cascades to its booking | `test_cancelling_a_slot_also_cancels_its_booking` | The clinic and patient views must agree. |
| Input validation | End before start, no timezone, past start, duration limits, bad email, empty name, unknown field, unknown status | Helpful 422 errors naming the field. |

## How the tests are built

- **Fake clock.** The app reads time from an injected function, so "the slot has started" is tested by moving the clock forward, with no sleeping.
- **Real SQLite file per test.** In-memory SQLite gives every connection its own empty database, which would make multi-threaded tests meaningless.
- **Proof the concurrency tests can fail.** We temporarily replaced the atomic claim with a naive check-then-write and removed the unique index. The tests failed (15 winners out of 20), then passed again once the real code was restored.
- **Stability.** The concurrency file passed 25 consecutive runs.

### tests/conftest.py

```
from datetime import datetime, timedelta, timezone

import pytest
from fastapi.testclient import TestClient

from app.main import create_app

START_OF_TIME = datetime(2030, 1, 1, 9, 0, tzinfo=timezone.utc)


class FakeClock:
    """Tests decide what 'now' is, so time-based rules need no sleeping."""
    def __init__(self):
        self.now = START_OF_TIME

    def __call__(self):
        return self.now


@pytest.fixture
def clock():
    return FakeClock()


@pytest.fixture
def app(tmp_path, clock):
    # A real file (not :memory:) so several threads and connections share one database.
    app = create_app(f"sqlite:///{tmp_path / 'test.db'}", clock=clock)
    yield app
    app.state.engine.dispose()


@pytest.fixture
def client(app):
    return TestClient(app)


@pytest.fixture
def clinic(client):
    return client.post("/clinics", json={"name": "Green Valley Clinic"}).json()


@pytest.fixture
def make_slot(client, clinic, clock):
    def _make(start=None, minutes=30):
        start = start or clock.now + timedelta(days=1)
        response = client.post(f"/clinics/{clinic['id']}/slots", json={
            "start_time": start.isoformat(),
            "end_time": (start + timedelta(minutes=minutes)).isoformat(),
        })
        assert response.status_code == 201, response.text
        return response.json()
    return _make


@pytest.fixture
def book(client):
    def _book(slot_id, name="Asha Rao", email="asha@example.com"):
        return client.post(f"/slots/{slot_id}/bookings",
                           json={"patient_name": name, "patient_email": email})
    return _book
```

### tests/test_concurrency.py

```
"""The most important tests: the booking rules must hold when requests overlap in time.

A threading.Barrier releases all workers together so their requests genuinely collide.
"""
import threading
from concurrent.futures import ThreadPoolExecutor
from datetime import timedelta

import pytest
from sqlalchemy import func, select, text
from sqlalchemy.exc import IntegrityError

from app.errors import ConflictError, DomainError
from app.models import Booking, BookingStatus, Slot, SlotStatus
from app.schemas import BookingCreate
from app.services import bookings, slots

WORKERS = 20


def run_together(app, jobs):
    """Run each job(session) in its own thread and session, all starting at the same instant."""
    barrier = threading.Barrier(len(jobs))
    results = [None] * len(jobs)

    def worker(index, job):
        with app.state.session_factory() as session:
            barrier.wait()
            try:
                results[index] = ("ok", job(session))
            except DomainError as error:
                results[index] = ("error", error.code)

    threads = [threading.Thread(target=worker, args=(i, job)) for i, job in enumerate(jobs)]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
    return results


def test_many_simultaneous_bookings_of_one_slot_produce_exactly_one_winner(app, make_slot, clock):
    slot = make_slot()

    def attempt(n):
        data = BookingCreate(patient_name=f"Patient {n}", patient_email=f"p{n}@example.com")
        return lambda session: bookings.book_slot(session, slot["id"], data, clock.now)
```

```
    results = run_together(app, [attempt(n) for n in range(WORKERS)])

    winners = [r for r in results if r[0] == "ok"]
    losers = [r for r in results if r[0] == "error"]
    assert len(winners) == 1
    assert len(losers) == WORKERS - 1
    assert {code for _, code in losers} == {"slot_already_booked"}

    with app.state.session_factory() as session:
        confirmed = session.scalar(select(func.count()).select_from(Booking).where(
            Booking.slot_id == slot["id"], Booking.status == BookingStatus.CONFIRMED))
        assert confirmed == 1


def test_simultaneous_http_requests_return_one_201_and_the_rest_409(client, make_slot):
    slot = make_slot()

    def attempt(n):
        return client.post(f"/slots/{slot['id']}/bookings",
                           json={"patient_name": f"P{n}", "patient_email": f"p{n}@example.com"}).status_code

    with ThreadPoolExecutor(max_workers=WORKERS) as pool:
        statuses = list(pool.map(attempt, range(WORKERS)))

    assert sorted(statuses) == [201] + [409] * (WORKERS - 1)


def test_booking_racing_against_slot_cancellation_never_leaves_inconsistent_state(app, make_slot, clock):
    """Whichever side wins, a cancelled slot must never keep a confirmed booking."""
    data = BookingCreate(patient_name="Asha Rao", patient_email="asha@example.com")
    slot_ids = [make_slot(start=clock.now + timedelta(days=1, hours=i))["id"] for i in range(15)]

    for slot_id in slot_ids:
        run_together(app, [
            lambda s, slot_id=slot_id: bookings.book_slot(s, slot_id, data, clock.now),
            lambda s, slot_id=slot_id: slots.cancel_slot(s, slot_id, clock.now),
        ])

    with app.state.session_factory() as session:
        for slot_id in slot_ids:
            slot = session.get(Slot, slot_id)
            confirmed = session.scalar(select(func.count()).select_from(Booking).where(
                Booking.slot_id == slot_id, Booking.status == BookingStatus.CONFIRMED))
            assert slot.status in (SlotStatus.CANCELLED, SlotStatus.BOOKED, SlotStatus.OPEN)
            assert confirmed == (1 if slot.status == SlotStatus.BOOKED else 0), (slot_id, slot.status)


def test_simultaneous_cancel_and_rebook_keep_slot_and_booking_in_step(app, make_slot, book, clock, client):
```

```
    """Cancel a booking while another patient tries to take the freed slot."""
    slot = make_slot()
    first = book(slot["id"], email="first@example.com").json()
    data = BookingCreate(patient_name="Second", patient_email="second@example.com")

    run_together(app, [
        lambda s: bookings.cancel_booking(s, first["id"], clock.now),
        lambda s: bookings.book_slot(s, slot["id"], data, clock.now),
    ])

    with app.state.session_factory() as session:
        status = session.get(Slot, slot["id"]).status
        confirmed = session.scalar(select(func.count()).select_from(Booking).where(
            Booking.slot_id == slot["id"], Booking.status == BookingStatus.CONFIRMED))
        # Either the rebook lost (slot stays booked until the cancel lands, then open) or won.
        assert confirmed == (1 if status == SlotStatus.BOOKED else 0)


def test_database_itself_refuses_a_second_confirmed_booking(app, make_slot, clock):
    """Defence in depth: even code that skips the service layer cannot double-book."""
    slot = make_slot()
    with app.state.session_factory() as session:
        for n in range(2):
            session.add(Booking(slot_id=slot["id"], patient_name="X", patient_email=f"x{n}@example.com",
                                created_at=clock.now))
        with pytest.raises(IntegrityError):
            session.commit()


def test_database_allows_a_new_booking_once_the_old_one_is_cancelled(app, make_slot, clock):
    slot = make_slot()
    with app.state.session_factory() as session:
        session.add(Booking(slot_id=slot["id"], patient_name="X", patient_email="x@example.com",
                            status=BookingStatus.CANCELLED, created_at=clock.now))
        session.add(Booking(slot_id=slot["id"], patient_name="Y", patient_email="y@example.com",
                            created_at=clock.now))
        session.commit()  # only the confirmed row counts toward the unique index
```

### tests/test_bookings.py

```
from datetime import timedelta


def test_booking_an_open_slot_succeeds_and_marks_it_booked(client, make_slot, book):
    slot = make_slot()
    response = book(slot["id"])
    assert response.status_code == 201
    booking = response.json()
    assert booking["status"] == "confirmed"
    assert booking["slot_id"] == slot["id"]
    assert client.get(f"/slots/{slot['id']}").json()["status"] == "booked"


def test_a_booked_slot_cannot_be_booked_again(client, clinic, make_slot, book):
    slot = make_slot()
    assert book(slot["id"], email="first@example.com").status_code == 201

    second = book(slot["id"], email="second@example.com")
    assert second.status_code == 409
    assert second.json()["error"]["code"] == "slot_already_booked"

    bookings = client.get(f"/clinics/{clinic['id']}/bookings").json()
    assert [b["patient_email"] for b in bookings] == ["first@example.com"]


def test_a_cancelled_slot_cannot_be_booked(client, make_slot, book):
    slot = make_slot()
    client.post(f"/slots/{slot['id']}/cancel")
    response = book(slot["id"])
    assert response.status_code == 409
    assert response.json()["error"]["code"] == "slot_cancelled"


def test_a_slot_that_has_started_cannot_be_booked(make_slot, book, clock):
    slot = make_slot()
    clock.now += timedelta(days=1, minutes=1)  # time passes; the slot has now started
    response = book(slot["id"])
    assert response.status_code == 409
    assert response.json()["error"]["code"] == "slot_in_past"


def test_booking_an_unknown_slot_is_404(book):
    response = book(4242)
    assert response.status_code == 404
    assert response.json()["error"]["code"] == "slot_not_found"


def test_booking_validates_name_and_email(client, make_slot):
```

```
    slot = make_slot()
    response = client.post(f"/slots/{slot['id']}/bookings",
                           json={"patient_name": "  ", "patient_email": "not-an-email"})
    assert response.status_code == 422
    fields = {d["field"] for d in response.json()["error"]["details"]}
    assert fields == {"patient_name", "patient_email"}
    assert client.get(f"/slots/{slot['id']}").json()["status"] == "open"  # nothing was claimed


def test_cancelling_a_booking_frees_the_slot_for_someone_else(client, make_slot, book):
    slot = make_slot()
    booking = book(slot["id"], email="first@example.com").json()

    cancelled = client.post(f"/bookings/{booking['id']}/cancel")
    assert cancelled.status_code == 200
    assert cancelled.json()["status"] == "cancelled"
    assert cancelled.json()["cancelled_at"] is not None
    assert client.get(f"/slots/{slot['id']}").json()["status"] == "open"

    assert book(slot["id"], email="second@example.com").status_code == 201


def test_a_booking_cannot_be_cancelled_twice(client, make_slot, book):
    booking = book(make_slot()["id"]).json()
    client.post(f"/bookings/{booking['id']}/cancel")
    again = client.post(f"/bookings/{booking['id']}/cancel")
    assert again.status_code == 409
    assert again.json()["error"]["code"] == "booking_already_cancelled"


def test_a_stale_cancel_does_not_reopen_a_slot_rebooked_by_someone_else(client, make_slot, book):
    slot = make_slot()
    first = book(slot["id"], email="first@example.com").json()
    client.post(f"/bookings/{first['id']}/cancel")
    book(slot["id"], email="second@example.com")

    client.post(f"/bookings/{first['id']}/cancel")  # replayed request for the old booking
    assert client.get(f"/slots/{slot['id']}").json()["status"] == "booked"


def test_a_booking_cannot_be_cancelled_after_its_slot_started(client, make_slot, book, clock):
    booking = book(make_slot()["id"]).json()
    clock.now += timedelta(days=2)
    response = client.post(f"/bookings/{booking['id']}/cancel")
    assert response.status_code == 409
    assert response.json()["error"]["code"] == "slot_in_past"
def test_cancelling_a_slot_also_cancels_its_booking(client, make_slot, book):
    slot = make_slot()
    booking = book(slot["id"]).json()

    assert client.post(f"/slots/{slot['id']}/cancel").status_code == 200
    assert client.get(f"/bookings/{booking['id']}").json()["status"] == "cancelled"
    assert client.get(f"/slots/{slot['id']}").json()["status"] == "cancelled"


def test_patient_can_list_their_own_bookings_case_insensitively(client, make_slot, book):
    mine = book(make_slot()["id"], email="Asha@Example.com").json()
    book(make_slot(start=None, minutes=45)["id"], email="other@example.com")

    response = client.get("/bookings", params={"patient_email": "asha@example.com"})
    assert [b["id"] for b in response.json()] == [mine["id"]]


def test_unknown_route_uses_the_same_error_shape(client):
    response = client.get("/nope")
    assert response.status_code == 404
    assert "error" in response.json()
```

### tests/test_slots.py

```
from datetime import timedelta

import pytest


def test_create_slot_returns_open_slot(client, clinic, clock):
    start = clock.now + timedelta(days=1)
    response = client.post(f"/clinics/{clinic['id']}/slots", json={
        "start_time": start.isoformat(), "end_time": (start + timedelta(minutes=30)).isoformat()})
    assert response.status_code == 201
    body = response.json()
    assert body["status"] == "open"
    assert body["clinic_id"] == clinic["id"]
    assert body["start_time"] == "2030-01-02T09:00:00Z"


def test_create_slot_rejects_end_before_start(client, clinic, clock):
    start = clock.now + timedelta(days=1)
    response = client.post(f"/clinics/{clinic['id']}/slots", json={
        "start_time": start.isoformat(), "end_time": (start - timedelta(minutes=30)).isoformat()})
    assert response.status_code == 422
    error = response.json()["error"]
    assert error["code"] == "validation_error"
    assert "end_time must be after start_time" in error["details"][0]["message"]


def test_create_slot_rejects_time_without_timezone(client, clinic):
    response = client.post(f"/clinics/{clinic['id']}/slots", json={
        "start_time": "2030-01-02T09:00:00", "end_time": "2030-01-02T09:30:00"})
    assert response.status_code == 422
    assert response.json()["error"]["details"][0]["field"] == "start_time"


def test_create_slot_rejects_start_in_the_past(client, clinic, clock):
    start = clock.now - timedelta(hours=1)
    response = client.post(f"/clinics/{clinic['id']}/slots", json={
        "start_time": start.isoformat(), "end_time": (start + timedelta(minutes=30)).isoformat()})
    assert response.status_code == 422
    assert response.json()["error"]["details"][0]["field"] == "start_time"


@pytest.mark.parametrize("minutes", [1, 9 * 60])
def test_create_slot_rejects_bad_duration(client, clinic, clock, minutes):
    start = clock.now + timedelta(days=1)
    response = client.post(f"/clinics/{clinic['id']}/slots", json={
        "start_time": start.isoformat(), "end_time": (start + timedelta(minutes=minutes)).isoformat()})
    assert response.status_code == 422
    assert "duration" in response.json()["error"]["details"][0]["message"]
```

```
def test_create_slot_for_unknown_clinic_is_404(client, clock):
    start = clock.now + timedelta(days=1)
    response = client.post("/clinics/999/slots", json={
        "start_time": start.isoformat(), "end_time": (start + timedelta(minutes=30)).isoformat()})
    assert response.status_code == 404
    assert response.json()["error"]["code"] == "clinic_not_found"


def test_unknown_field_is_rejected(client, clinic, clock):
    start = clock.now + timedelta(days=1)
    response = client.post(f"/clinics/{clinic['id']}/slots", json={
        "start_time": start.isoformat(), "end_time": (start + timedelta(minutes=30)).isoformat(),
        "doctor": "Dr Rao"})
    assert response.status_code == 422


def test_list_slots_can_filter_by_status_and_is_ordered_by_time(client, clinic, make_slot, book, clock):
    later = make_slot(start=clock.now + timedelta(days=3))
    sooner = make_slot(start=clock.now + timedelta(days=1))
    book(sooner["id"])

    everything = client.get(f"/clinics/{clinic['id']}/slots").json()
    assert [s["id"] for s in everything] == [sooner["id"], later["id"]]

    open_only = client.get(f"/clinics/{clinic['id']}/slots", params={"status": "open"}).json()
    assert [s["id"] for s in open_only] == [later["id"]]


def test_list_slots_rejects_unknown_status(client, clinic):
    response = client.get(f"/clinics/{clinic['id']}/slots", params={"status": "pending"})
    assert response.status_code == 422


def test_cancel_slot_then_cancel_again_conflicts(client, make_slot):
    slot = make_slot()
    first = client.post(f"/slots/{slot['id']}/cancel")
    assert first.status_code == 200
    assert first.json()["status"] == "cancelled"

    second = client.post(f"/slots/{slot['id']}/cancel")
    assert second.status_code == 409
    assert second.json()["error"]["code"] == "slot_already_cancelled"


def test_cancel_unknown_slot_is_404(client):
    assert client.post("/slots/12345/cancel").status_code == 404
```

### pytest.ini

```
[pytest]
testpaths = tests
filterwarnings =
    ignore:Using `httpx` with `starlette.testclient`:Warning
```

# 9. README: setup, run and test

Use this as `README.md` in the repository root. It is the file reviewers read first, so it covers setup, running, endpoints and tests in the order they need them.

### README.md

```
# Clinic Booking API

Clinics publish time slots; patients book them. A slot can never be double-booked,
even when two requests arrive at the same moment.

Built with Python, FastAPI, SQLAlchemy. SQLite by default, PostgreSQL supported.

## Setup

Requires Python 3.11 or newer.

    python -m venv .venv
    source .venv/bin/activate        # Windows: .venv\Scripts\activate
    pip install -r requirements.txt

## Run

    uvicorn app.main:create_app --factory --reload

The API is at http://127.0.0.1:8000 and interactive docs are at http://127.0.0.1:8000/docs.
The SQLite database is created automatically as ./clinic.db.

To use PostgreSQL instead (also run `pip install "psycopg[binary]"`):

    export DATABASE_URL=postgresql+psycopg://user:password@localhost/clinic

## Run the tests

    pytest

31 tests run in about two seconds. They use a temporary SQLite file and a fake clock,
so they need no running server and no setup.

## Endpoints

| Method | Path                           | Purpose                                          |
|--------|--------------------------------|--------------------------------------------------|
| POST   | /clinics                       | Create a clinic                                  |
| GET    | /clinics                       | List clinics                                     |
| GET    | /clinics/{id}                  | Get a clinic                                     |
| POST   | /clinics/{id}/slots            | Publish a slot                                   |
| GET    | /clinics/{id}/slots            | List slots (status, starts_after, starts_before, limit, offset) |
| GET    | /clinics/{id}/bookings         | List a clinic's bookings                         |
| GET    | /slots/{id}                    | Get a slot                                       |
| POST   | /slots/{id}/cancel             | Cancel a slot (also cancels its booking)         |
| POST   | /slots/{id}/bookings           | Book a slot                                      |
| GET    | /bookings?patient_email=...    | List a patient's bookings                        |
| GET    | /bookings/{id}                 | Get a booking                                    |
```

```
| POST   | /bookings/{id}/cancel          | Cancel a booking (re-opens the slot)      |

## Booking rules

- A slot has at most one active booking, even under simultaneous requests (409 slot_already_booked).
- A cancelled slot cannot be booked (409 slot_cancelled).
- A slot that has started cannot be booked (409 slot_in_past).
- Cancelling a booking re-opens the slot. Cancelling a slot cancels its booking.
- Slots must start in the future, end after they start, last 5 minutes to 8 hours,
  and use times with a timezone (for example 2030-03-01T09:00:00Z).

## Example requests

    curl -X POST localhost:8000/clinics -H 'Content-Type: application/json' \
      -d '{"name":"Green Valley Clinic"}'

    curl -X POST localhost:8000/clinics/1/slots -H 'Content-Type: application/json' \
      -d '{"start_time":"2030-03-01T09:00:00Z","end_time":"2030-03-01T09:30:00Z"}'

    curl 'localhost:8000/clinics/1/slots?status=open'

    curl -X POST localhost:8000/slots/1/bookings -H 'Content-Type: application/json' \
      -d '{"patient_name":"Asha Rao","patient_email":"asha@example.com"}'

    curl -X POST localhost:8000/bookings/1/cancel

## Errors

Every error looks like:

    {"error": {"code": "slot_already_booked", "message": "This slot has already been booked."}}

404 not found, 409 conflict with current state, 422 invalid input (with a `details`
list naming each bad field).

## How double-booking is prevented

Booking claims the slot with one atomic statement:

    UPDATE slots SET status='booked' WHERE id=:id AND status='open' AND start_time > :now

Only one of any number of simultaneous requests changes a row. A partial unique index on
confirmed bookings is a second guard at the database level. See tests/test_concurrency.py.

## Project layout

    app/routers/    HTTP only
    app/services/   business rules
    app/models.py   tables and constraints
    app/schemas.py  request/response validation
    tests/          booking rules and concurrency tests
```

### To publish on GitHub

1. Create the files from sections 6 and 8 exactly as shown, plus empty `__init__.py` files.
2. Add a `.gitignore` containing `.venv/`, `__pycache__/`, `*.db`, `*.db-wal`, `*.db-shm`.
3. Run `pytest` once locally and confirm 31 tests pass.
4. Commit and push, then paste the repository link into the submission form.

# 10. Trade-offs, limitations and next steps

The scope matches the brief: correct booking rules first, then clean structure and tests. These are the gaps a reviewer may ask about, and the planned fix for each.

| Limitation | Impact | Next step |
|---|---|---|
| PostgreSQL was not run where this was built | SQLite behaviour is tested (including 25 repeated concurrency runs). PostgreSQL behaviour rests on documented row-lock semantics, not a test run. | Run `pytest` against Postgres by setting `DATABASE_URL`, ideally as a CI job with a Postgres service container. |
| No authentication | Anyone can cancel any slot or booking. The brief has no accounts, so this was left out deliberately. | Staff tokens for slot endpoints; booking cancellation tied to the patient who made it. |

| Limitation | Impact | Next step |
|---|---|---|
| Overlapping slots are allowed | A clinic with several practitioners legitimately has parallel slots, and the brief has no practitioner field. | Add `practitioner_id` and, on PostgreSQL, an exclusion constraint on (`practitioner_id`, time range). |
| No idempotency key on booking | If a client retries a booking whose first attempt succeeded, the retry gets 409. | Accept an `Idempotency-Key` header and return the original booking for repeats. |
| Tables are created at startup | Schema changes need manual handling. | Alembic migrations. |
| One patient can hold overlapping bookings at different clinics | Possible double-scheduling for one person. | Optional rule or warning, decided with the product owner. |
| SQLite allows one writer at a time | Fine for a small deployment; throughput is limited. | Use PostgreSQL in production, which this design already supports. |
| Offset pagination | Slightly slow for very deep pages. | Keyset pagination on (`start_time`, `id`). |

## Summary of the "why"

- The **atomic conditional update** exists because "check then write" has a gap that simultaneous requests fall into; we saw 15 of 20 requests win one slot with the naive version.
- The **partial unique index** exists so that a future coding mistake cannot silently corrupt bookings.
- **Services separate from routers** exist so rules can be tested without HTTP and reused later.
- The **injected clock** and **per-test database** exist so time-based and concurrent rules can be tested reliably.
- **One error format with stable codes** exists so clients can react to a specific failure instead of parsing text.